Fast Tournament Equity Computation via Generating-Function Quadrature and FFT-Accelerated Subproduct Trees

Sam Rosenstrauch
sarose550@gmail.com

July 21, 2026

Abstract

The Independent Chip Model (ICM) maps poker tournament chip stacks to expected prize money, but exact computation requires summing over all $n!$ elimination orderings. I present a deterministic algorithm that reformulates ICM equity as a one-dimensional integral over generating-function coefficients, evaluated by Gaussian quadrature ($Q = 256$ nodes, relative error $< 1.6 \times 10^{-10}$). The central challenge - computing leave-one-out polynomial products for all n players - is solved by an FFT-accelerated subproduct tree whose propagation phase is the adjoint of its build phase, reducing cost from $\mathcal{O}(nk)$ to $\mathcal{O}(n \log^2 k)$ per quadrature point. An empirical lookup table (calibrated per device) dispatches automatically between a batched linear engine and a hybrid block-tree engine. I evaluate the CPU implementation in detail on Apple M3 Pro and AMD Zen 4 (with AOCL-FFTW): the hybrid and linear engines compute exact equities for up to 17,984 players in one second, single-threaded, on Zen 4. A CUDA implementation for NVIDIA B200 extends this to roughly 1.5 million players in about a second.

1 Introduction

Multi-table poker tournaments are structured competitions in which players begin with equal chip stacks and are progressively eliminated as they lose their chips. A payout structure $\pi = (\pi_0, \pi_1, \ldots, \pi_{k-1})$ specifies the prize awarded to each finishing position, with $k \leq n$ positions receiving nonzero prizes and $\pi_0 \geq \pi_1 \geq \cdots \geq \pi_{k-1} \geq 0$ (position $r+1$ receives payout π_r). The *Independent Chip Model* (ICM) provides a principled mapping from a player's chip stack to their expected tournament equity - the expected prize money they will receive [Malmuth, 1987, Harville, 1973].

Despite its central role in tournament strategy, exact ICM computation is fundamentally difficult because the number of possible elimination orderings grows factorially in n. For $n = 20$ there are 2.4×10^{18} orderings to consider; for $n = 30$, the count reaches 2.7×10^{32}. Bitmask dynamic programming reduces this to $\mathcal{O}(n \cdot 2^n)$ time by iterating over subsets of surviving players [Malmuth, 1987], but the exponential state space limits practical computation to roughly $n \leq 23$. Monte Carlo sampling provides approximate solutions for larger fields [ICMIZER, 2024], but error shrinks only as $\mathcal{O}(1/\sqrt{N})$, making high-precision results prohibitively expensive.

The key observation that enables a better approach is that each player's ICM equity can be expressed as a one-dimensional integral rather than an exponential sum. This follows from the *exponential clock framing*, where each player is assigned an independent exponential random variable

and players are eliminated in order of increasing firing time. Under this framing, the combinatorial sum collapses into an integral that depends only on the chip stacks.

The integral is evaluated by Gaussian quadrature with $Q = 256$ nodes, achieving exponential convergence to double-precision accuracy - a choice driven by worst-case stack ratios up to 10^9, not just the well-behaved uniform case (Section 6.3 justifies the specific value). The remaining challenge is evaluating the integrand efficiently at each quadrature point, which requires computing a degree-k leave-one-out polynomial product for each of n players. I address this with an FFT-accelerated binary subproduct tree whose propagation phase is the adjoint of the build phase - a cross-correlation that distributes the payout vector to all leaves simultaneously.

My contributions are:

1. A rigorous derivation of ICM equities as coefficient extractions from a generating function, evaluated via numerical quadrature (Sections 3–4).

2. A hybrid engine combining a subproduct tree for inter-block computation with direct polynomial arithmetic for intra-block computation, with correctness proved via the adjoint relationship between convolution and cross-correlation (Section 4).

3. An empirical crossover table, calibrated offline via direct timing on the target hardware, that achieves 100% correct engine dispatch on all serial bench_grid cells and 83/84 on parallel cells across both platforms, replacing earlier analytical cost models (Section 4.5).

4. Platform-specific implementations for ARM64 (Apple M3 Pro), x86-64 (AMD Zen 4 with AOCL-FFTW), and CUDA (NVIDIA B200 GPU), demonstrating the approach at scale (Sections 6, 7, and 8).

2 Related Work

ICM and the Malmuth–Harville model. The probability model underlying ICM was introduced by Harville [1973] for horse-racing handicapping and adapted to poker tournaments by Malmuth [1987]. Under this model, the probability that any surviving player is eliminated next is proportional to their chip stack. The standard exact algorithm uses bitmask dynamic programming over all 2^n subsets of surviving players, running in $\mathcal{O}(n \cdot 2^n)$ time. For large fields, Monte Carlo sampling is the prevailing approach, with error converging as $\mathcal{O}(1/\sqrt{N})$.

Exponential clock formulation. The equivalence between the Malmuth–Harville elimination process and independent exponential clocks was noted by Tysen Streib in a 2011 forum post [Streib, 2011], who used it to develop an efficient Monte Carlo sampling method. The key insight is that assigning each player an $\mathrm{Exp}(S_j)$ random variable and sorting by realized value produces an elimination order whose distribution matches the sequential-elimination model exactly, while requiring only n random draws and one sort per sampled tournament.

Polynomial arithmetic and subproduct trees. The subproduct tree (also called a *subresultant tree* or *product tree*) is a standard tool in computer algebra for multi-point evaluation and interpolation. von zur Gathen and Gerhard [2013] provide a comprehensive treatment, including the $\mathcal{O}(n \log^2 n)$ algorithms for computing the product of n linear factors and evaluating a polynomial at n points. My use of the tree is closest to the "accumulating remainder" formulation, but with the critical difference that propagation computes cross-correlations (the adjoint of convolution)

rather than polynomial remainders. This adjoint perspective, proved in Theorem 4.4, appears to be new in this context.

FFT libraries and automatic tuning. All FFT-accelerated engines in this work build on the Cooley–Tukey algorithm [Cooley and Tukey, 1965], which reduces the DFT from $\mathcal{O}(d^2)$ to $\mathcal{O}(d \log d)$. FFTW [Frigo and Johnson, 2005] introduced the idea of empirical auto-tuning for FFT plans, generating machine-specific "codelets" at install time. AMD's AOCL-FFTW extends this with znver4-tuned AVX-512-specific codelets (versus the generic upstream distribution) [Advanced Micro Devices, Inc., 2024]. I use AOCL-FFTW as the sole FFT backend on Zen 4 after a direct A/B measurement against plain system FFTW3 (Section 6) showed a clean, reproducible win at the kernel level with no sizes where the reverse held ; so no dual-library dispatch is warranted on this platform, unlike the GPU's cuFFT/cuFFTDx split (Section 8). The broader philosophy of empirically-tuned numerical libraries traces to FFTW's `PATIENT` planner [Frigo and Johnson, 2005] and SPIRAL [Püschel et al., 2005]. The approach of searching a parameter space empirically per architecture, as done here for the (n, k)-indexed crossover and block-size tables, is closest in spirit to the register-blocking search in the BeBOP/Sparsity system [Demmel et al., 2005], which times candidate register-blocking and unrolling parameters for sparse matrix kernels.

Roofline performance models. The roofline model of Williams et al. [2009] bounds achievable performance by the minimum of peak compute and memory bandwidth, parameterized by arithmetic intensity. An earlier version of the engine dispatch (Section 4.5) used a roofline model with harmonic-mean bandwidth blending across cache levels; this was superseded by the current empirical crossover table after the analytical model proved unable to match measured crossover points on real hardware.

GPU FFT and CUDA Graphs. NVIDIA's cuFFT library supports batched FFT execution, and the cuFFTDx extensions [NVIDIA Corporation, 2024b] enable fused device-side FFTs that eliminate intermediate memory round-trips. CUDA Graphs [NVIDIA Corporation, 2024a] amortize kernel launch overhead by capturing and replaying entire execution graphs, which is essential for the dozens of kernel launches in a single tree traversal.

3 Mathematical Derivation

This section derives the integral representation that is the foundation of the algorithm. All of the key ideas - the exponential clock, the generating function, and the one-dimensional integral - are presented here, with supporting proofs where they provide insight.

3.1 The Malmuth–Harville Model and Exponential Clocks

Definition 3.1 (Tournament instance)**.** *A* tournament instance *is a tuple* (n, S, π) *where* $n \in \mathbb{N}$ *is the number of players,* $S = (S_1, \ldots, S_n) \in \mathbb{R}^n_{>0}$ *is the vector of chip stacks with* $S_i > 0$ *for all* i*, and* $\pi = (\pi_0, \ldots, \pi_{k-1}) \in \mathbb{R}^k_{\geq 0}$ *is the payout vector with* $k \leq n$ *and* $\pi_0 \geq \pi_1 \geq \cdots \geq \pi_{k-1} \geq 0$ *(0-indexed: position* $r + 1$ *receives payout* π_r*).*

Definition 3.2 (Malmuth–Harville model)**.** *Under the Malmuth–Harville (MH) model, the probability that player* i *is the first to be eliminated from a set* $T \subseteq \{1, \ldots, n\}$ *of surviving players*

is:

$$\Pr[i \textit{ elim. first} \mid T] = \frac{S_i}{\sum_{j \in T} S_j}\,. \tag{1}$$

Elimination proceeds sequentially: after the first player is eliminated, the model is applied recursively to the remaining set $T \setminus \{i\}$ with unchanged stacks.

The MH model has an equivalent continuous-time formulation that is central to my approach. Assign each player j an independent exponential random variable $T_j \sim \text{Exp}(S_j)$ with density $S_j e^{-S_j t}$ for $t \geq 0$. Players are eliminated in order of increasing T_j - smallest T_j is eliminated first, etc.

Why does this reproduce the MH model? For any two players i, j, the probability that i has a smaller clock than j is:

$$\Pr[T_i < T_j] = \frac{S_i}{S_i + S_j}.$$

More generally, for any set of surviving players, the memoryless property of the exponential distribution guarantees that the conditional probability of any surviving player being eliminated next is proportional to their stack, recovering Equation 1 exactly.

This means we can sample an entire elimination order in one shot: draw n independent exponentials, sort by their realized values, and the resulting order is distributed according to the MH model. This is the basis for efficient Monte Carlo ICM methods, but it also enables something stronger.

3.2 The Generating Function

Fix player i and condition on $T_i = t$. For each other player $j \neq i$, we have, independently:

- j finishes *before* i iff $T_j < t$, which happens with probability $b_j(t) = \Pr[T_j < t] = 1 - e^{-S_j t}$.
- j finishes *after* i with probability $a_j(t) = \Pr[T_j > t] = e^{-S_j t}$.

Because the clocks are independent, the probability of any specific pattern of "who finishes before i" is the product of the individual probabilities. This product structure suggests a generating function. Define:

$$Q_i(x; t) = \prod_{j \neq i} \big(a_j(t) + b_j(t) \cdot x\big). \tag{2}$$

Expanding this product, each factor contributes either $a_j(t)$ (player j finishes after i, contributing x^0) or $b_j(t) \cdot x$ (player j finishes before i, contributing x^1). The coefficient of x^r in $Q_i(x; t)$ is therefore exactly the probability that exactly r of the other players finish before i, conditioned on $T_i = t$:

$$[x^r]\, Q_i(x; t) = \Pr[i \text{ finishes in position } r + 1 \mid T_i = t]. \tag{3}$$

3.3 The Integral Representation

To uncondition, we integrate against the density of T_i:

$$\Pr[i \text{ finishes in position } r + 1] = \int_0^\infty S_i e^{-S_i t} \cdot [x^r]\, Q_i(x; t)\, dt. \tag{4}$$

Now change variables: let $v = e^{-t}$, so $dv = -e^{-t}\,dt = -v\,dt$ and $dt = -dv/v$. Substituting $a_j(t) = e^{-S_j t} = v^{S_j}$ and $b_j(t) = 1 - v^{S_j}$:

$$Q_i(x; v) = \prod_{j \neq i} \big(v^{S_j} + (1 - v^{S_j})x\big). \tag{5}$$

As t goes from 0 to ∞, v goes from 1 to 0. The minus sign from $dt = -dv/v$ cancels the bound flip, yielding:

$$\Pr[i \text{ finishes in position } r+1] = \int_0^1 S_i\, v^{S_i - 1} \cdot [x^r]\, Q_i(x; v)\, dv. \tag{6}$$

Multiplying by the payout π_r and summing over positions:

$$E_i = \int_0^1 S_i\, v^{S_i - 1} \underbrace{\sum_{r=0}^{k-1} \pi_r \cdot [x^r]\, Q_i(x; v)}_{=:\, \Psi_i(v)}\, dv. \tag{7}$$

The inner sum $\Psi_i(v) = \sum_{r=0}^{k-1} \pi_r \cdot [x^r] Q_i(x; v)$ is the dot product of the payout vector with the coefficient vector of Q_i - exactly the quantity the subproduct tree will compute efficiently.

3.4 Numerical Quadrature

The integral in Equation 7 is over $v \in (0, 1)$. For numerical integration, I apply the change of variables $v = \Phi(y)$ where Φ is the standard normal CDF. This maps $(0, 1)$ to $\mathbb{R}$ and produces an integrand that decays like $e^{-y^2/2}$ as $|y| \to \infty$, making it ideal for the trapezoidal rule [Trefethen and Weideman, 2014]:

$$E_i \approx \sum_{q=1}^{Q} w_q \cdot S_i \cdot v_q^{S_i - 1} \cdot \Psi_i(v_q), \tag{8}$$

where $(v_q, w_q)_{q=1}^{Q}$ are the quadrature nodes and weights.

This rule (which I call *erfc-trap*) achieves exponential convergence: the error shrinks roughly as e^{-cQ} for some $c > 0$ depending on the strip of analyticity of the integrand. With $Q = 256$, relative error is below 5×10^{-12} on uniform stacks and below 1.6×10^{-10} even at the 10^9 stack-ratio bound (see Section 6.3 for a detailed comparison against tanh-sinh quadrature).

4 Algorithm

The quadrature rule reduces ICM computation to evaluating $\Psi_i(v_q) = \sum_{r=0}^{k-1} \pi_r \cdot [x^r] Q_i(x; v_q)$ for all $i = 1, \ldots, n$ at each of Q quadrature points. Computing $Q_i(x; v_q) = \prod_{j \neq i}(a_j + b_j x)$ naively - one player at a time - would cost $\mathcal{O}(n^2 k)$ per quadrature point. This section presents three engines that compute all n leave-one-out products simultaneously, with cost-model-driven automatic dispatch.

I will work throughout with truncated polynomials in the vector space $\mathbb{R}[x]_{<k} \cong \mathbb{R}^k$ of polynomials of degree $< k$, equipped with the Euclidean inner product $\langle f, g \rangle = \sum_{m=0}^{k-1} f_m g_m$, so that $\Psi_i(v) = \langle \pi, Q_i \rangle$.

4.1 Engine 1: Batched Linear Forward-Backward

The linear engine computes all n inner products in $\mathcal{O}(nk)$ time using a forward-backward decomposition. It is optimal when k is small.

Algorithm 1: Linear Forward-Backward

Input: n players, values $a_1, \ldots, a_n$, 0-indexed payout $\pi \in \mathbb{R}^k$
Output: $\Psi_1, \ldots, \Psi_n$

`// Forward: prefix products`
$g^{(0)} \leftarrow \pi$
for $j = 1, \ldots, n$ **do**
  for $m = 0, \ldots, k-2$ **do**
    $g_m^{(j)} \leftarrow a_j \cdot g_m^{(j-1)} + (1 - a_j) \cdot g_{m+1}^{(j-1)}$
  end
  $g_{k-1}^{(j)} \leftarrow a_j \cdot g_{k-1}^{(j-1)}$
end
`// Backward: suffix products, extract inner products`
$(R_0, \ldots, R_{k-1}) \leftarrow (1, 0, \ldots, 0)$
for $j = n, n-1, \ldots, 1$ **do**
  $\Psi_j \leftarrow \sum_{m=0}^{k-1} g_m^{(j-1)} \cdot R_m$
  for $m = k-1, k-2, \ldots, 1$ **do**
    $R_m \leftarrow a_j \cdot R_m + (1 - a_j) \cdot R_{m-1}$
  end
  $R_0 \leftarrow a_j \cdot R_0$
end
return $\Psi_1, \ldots, \Psi_n$

The forward pass computes prefix products $g^{(j)} = \pi * P_1 * \cdots * P_j \bmod x^k$, where $P_j(x) = a_j + (1 - a_j)x$. The backward pass constructs suffix products $R = P_{j+1} * \cdots * P_n \bmod x^k$ incrementally, extracting $\Psi_j = \langle g^{(j-1)}, R \rangle$ at each step. The inner product and suffix update are fused into a single loop because the backward iteration over m reads R_m for the dot product before overwriting it.

Batched quadrature and SIMD. To exploit SIMD parallelism, I interleave $B_Q = 8$ quadrature points in the data layout: $a_{\text{batch}}[j \cdot B_Q + q_i]$ stores the q_i-th quadrature value for player j, ensuring sequential access in the inner loops. $B_Q = 8$ wins on all platforms: on Zen 4 it maps to one native 512-bit AVX-512 vector per player, and on Apple M3 Pro it maps to two NEON FMA operations spread across 4 FMA ports.

L2-aware checkpointing. When $n \cdot k \cdot B_Q \cdot 8$ bytes exceeds L2 cache, the forward pass saves checkpoints every $C = \lfloor L_2/(k \cdot B_Q \cdot 8) \rfloor$ players. The backward pass recomputes each segment from its checkpoint before extracting equities, trading $\mathcal{O}(n/C)$ recomputation passes for $\mathcal{O}(C \cdot k)$ memory. This is critical not only for the serial engine but also for the parallel (OpenMP) path, where omitting it caused a 13.5× regression by forcing DRAM bandwidth instead of L2.

4.2 Engine 2: FFT-Accelerated Subproduct Tree

For large k, the linear engine's $\mathcal{O}(nk)$ cost becomes expensive. The tree engine replaces it with a $\mathcal{O}(n \log^2 k)$ approach based on a binary subproduct tree.

Definition 4.1 (Binary subproduct tree)**.** *Given n linear factors $P_j(x) = a_j + (1 - a_j)x$ for $j = 1, \ldots, n$, the binary subproduct tree has $L = \lceil \log_2 n \rceil + 1$ levels. Level 0 contains the n leaves P_j. Level ℓ contains $\lceil n/2^\ell \rceil$ nodes, each being the product of its two children (truncated to degree k):*

$$P_i^{(\ell)} = P_{2i}^{(\ell-1)} \cdot P_{2i+1}^{(\ell-1)} \bmod x^k.$$

Algorithm 2: Tree Build and Propagate

Input: Leaf polynomials $P_1^{(0)}, \ldots, P_n^{(0)}$, 0-indexed payout π, degree k
Output: $\Psi_1, \ldots, \Psi_n$

`// Build (bottom-up)`
for $\ell = 1, \ldots, L-2$ **do**
  for *each node i at level ℓ* ***in parallel*** **do**
    $P_i^{(\ell)} \leftarrow P_{2i}^{(\ell-1)} \cdot P_{2i+1}^{(\ell-1)} \bmod x^k$
  end
end

`// Propagate (top-down)`
$g^{(L-1)} \leftarrow \pi$
for $\ell = L-1, L-2, \ldots, 1$ **do**
  for *each node i at level ℓ* ***in parallel*** **do**
    $g_{2i}^{(\ell-1)} \leftarrow g_i^{(\ell)} \star P_{2i+1}^{(\ell-1)}$ `// Left child via right sibling`
    $g_{2i+1}^{(\ell-1)} \leftarrow g_i^{(\ell)} \star P_{2i}^{(\ell-1)}$ `// Right child via left sibling`
  end
end
return $g_1^{(0)}[0], \ldots, g_n^{(0)}[0]$

4.3 The Adjoint: Why Propagation Works

The correctness of the tree engine follows from a single algebraic fact: the adjoint of truncated convolution is cross-correlation.

Definition 4.2. *For $h \in \mathbb{R}[x]_{<k}$, let $T_h : \mathbb{R}[x]_{<k} \to \mathbb{R}[x]_{<k}$ denote truncated convolution by h: $T_h(f) = f * h$, i.e. $(T_h(f))_m = \sum_{i+j=m} f_i h_j$. Define cross-correlation by f and h as $(f \star h)_j = \sum_{m=0}^{k-1-j} h_m f_{m+j}$.*

Lemma 4.3 (Correlation is the adjoint of convolution)**.** *For all $f, g, h \in \mathbb{R}[x]_{<k}$, $\langle T_h(f),\, g \rangle = \langle f,\, g \star h \rangle$; equivalently, $T_h^* = \cdot \star h$.*

Proof. T_h has a lower-triangular Toeplitz matrix representation $M_{ij} = h_{i-j}$ (with $h_\ell = 0$ for $\ell < 0$ or $\ell \geq k$). Then $\langle Mf,\, g \rangle = f^\top M^\top g = \langle f,\, M^\top g \rangle$, and $(M^\top g)_i = \sum_{j=0}^{k-1} h_{j-i} g_j = \sum_{m=0}^{k-1-i} h_m g_{m+i} = (g \star h)_i$. □

The build phase constructs the full product $R = P_1 * \cdots * P_n$ via a binary tree of truncated convolutions. Reading from root to leaf j, the product decomposes as $R = Q_j * P_j \bmod x^k$, where Q_j is the product of all *sibling* polynomials on the path from leaf j to the root.

The propagation phase computes $\langle \pi,\, Q_j \rangle$ for all j by applying the adjoint of the build phase's linear map, with π as the seed. At each internal node, the build phase computes $(L, R) \mapsto L * R \bmod x^k$. Holding the sibling fixed and viewing this as the linear map T_S (where S denotes the sibling polynomial) in the descendant of interest, the adjoint T_S^* is $g \mapsto g \star S$ - exactly what Algorithm 2 implements at each level.

Theorem 4.4 (Correctness of the tree engine)**.** *After Algorithm 2 completes,* $g_j^{(0)}[0] = \Psi_j$ *for all* $j = 1, \ldots, n$.

Proof. We establish the invariant by downward induction on ℓ. Let $\mathrm{leaves}(i, \ell)$ be the leaf indices in the subtree rooted at node i at level ℓ, and let $C_j^{(\ell)}$ be the set of all leaves *outside* this subtree. The invariant is:

$$g_i^{(\ell)} = \pi \star \Big(\prod_{j \in C_j^{(\ell)}} P_j \bmod x^k \Big). \tag{9}$$

Base ($\ell = L{-}1$, root): $C_j^{(L-1)} = \emptyset$, and $\pi \star 1 = \pi = g^{(L-1)}$.

Inductive step: Assume the invariant holds at level ℓ for node i. The left child receives $g_{2i}^{(\ell-1)} = g_i^{(\ell)} \star P_{2i+1}^{(\ell-1)}$. For any test vector f:

$$\begin{aligned}
\langle f,\, g_{2i}^{(\ell-1)} \rangle &= \langle f,\, (\pi \star R_C) \star P_{2i+1} \rangle \\
&= \langle f * P_{2i+1},\, \pi \star R_C \rangle \quad \text{(Lemma 4.3)} \\
&= \langle (f * P_{2i+1}) * R_C,\, \pi \rangle \quad \text{(Lemma 4.3)} \\
&= \langle f * (P_{2i+1} * R_C),\, \pi \rangle \quad \text{(associativity)} \\
&= \langle f,\, \pi \star (P_{2i+1} * R_C) \rangle \quad \text{(Lemma 4.3).}
\end{aligned}$$

Since this holds for all f, the invariant holds for the left child with $C_j^{(\ell-1)} = C_j^{(\ell)} \cup \mathrm{leaves}(2i{+}1, \ell{-}1)$. The right child is symmetric. At level 0, $C_j^{(0)} = [n] \setminus \{j\}$, so $g_j^{(0)} = \pi \star Q_j$ and $g_j^{(0)}[0] = \langle \pi,\, Q_j \rangle = \Psi_j$. □

Remark 4.5. *The propagation phase is an instance of reverse-mode automatic differentiation applied to the build phase [Griewank and Walther, 2008]. Writing* T_ℓ *for the convolution map* T_h *of Lemma 4.3 instantiated with* h *equal to level* ℓ*'s sibling polynomial, the build computes* $T_L \circ \cdots \circ T_1$*, and propagation computes* $T_1^* \circ \cdots \circ T_L^*$ *applied to* π*. A standard result in reverse-mode AD states that the adjoint of a linear program has the same arithmetic complexity, giving complexity parity between build and propagation for free.*

Per-level FFT and paired cached correlate. At each tree level, I compare calibrated FFT costs against schoolbook multiplication to decide whether to use FFT. FFT sizes are chosen from all 7-smooth numbers ($2^a 3^b 5^c 7^d$) to minimize total cost including wrap correction. During propagation, I share the forward FFT of the parent g-vector across both child correlates and cache child frequency-domain representations from the build phase, halving the FFT count at each level.

This sharing is possible because correlation and convolution differ only by a conjugation in the frequency domain: for real-valued f, h, the discrete Fourier transform satisfies $\widehat{f \star h} = \hat{f} \cdot \overline{\hat{h}}$, whereas ordinary convolution satisfies $\widehat{f * h} = \hat{f} \cdot \hat{h}$ (no conjugate). Conjugating a transformed vector is

$\mathcal{O}(d)$ - an elementwise sign flip on the imaginary part - versus $\mathcal{O}(d \log d)$ for a fresh transform. Two consequences follow directly. First, each child polynomial's forward FFT, already computed once during the build phase to convolve it with its sibling (Algorithm 2, build step), needs only to be conjugated - not retransformed - to serve as that child's correlation kernel during propagation; this is the "cached" frequency-domain representation. Second, propagation at a node correlates the *same* parent g-vector against both children's (cached, conjugated) spectra ($g_i^{(\ell)} \star P_{2i+1}^{(\ell-1)}$ and $g_i^{(\ell)} \star P_{2i}^{(\ell-1)}$), so a single forward FFT of $g_i^{(\ell)}$ - the "paired" share - suffices for both child updates instead of transforming $g_i^{(\ell)}$ separately for each, halving the FFT count at this level relative to a naive implementation.

4.4 Engine 3: Hybrid Block-Divide

The hybrid engine partitions the n players into $\lceil n/B \rceil$ blocks of size B and combines direct $\mathcal{O}(B^2)$ within-block computation with the tree engine for inter-block structure.

Algorithm 3: Hybrid Block-Divide

Input: n players, values $a_1, \ldots, a_n$, 0-indexed payout π, degree k, block size B
Output: $\Psi_1, \ldots, \Psi_n$

`// Step 1: Build block products`
for *each block* $b = 0, 1, \ldots, \lceil n/B \rceil - 1$ **do**
  $P_b(x) \leftarrow \prod_{j \in \text{block } b}(a_j + (1 - a_j)x)$ `// Sequential,` $\mathcal{O}(B^2)$
end

`// Step 2: Inter-block tree (Algorithm 2) with block leaves`
Build subproduct tree from block products $P_0, \ldots, P_{\lceil n/B \rceil - 1}$
Propagate π to obtain block-level g-vectors g_b

`// Step 3: Within-block divide`
for *each block* b **do**
  for *each player* j *in block* b **do**
    Compute $Q_j^{(b)} \leftarrow P_b(x)/(a_j + (1 - a_j)x)$ via stable bidirectional synthetic division
    $\Psi_j \leftarrow \sum_{m=0}^{\min(k,B)-1} g_{b,m} \cdot Q_{j,m}^{(b)}$
  end
end
return $\Psi_1, \ldots, \Psi_n$

Bidirectional division stability. Write the complete block product as $P_b(x) = \sum_{m=0}^{B} P_m x^m$ and the quotient to be recovered as $Q_j^{(b)}(x) = \sum_{m=0}^{B-1} Q_m x^m$, so that $P_b = (a_j + (1 - a_j)x) \cdot Q_j^{(b)}$. Matching coefficients on both sides gives two exact recurrences for Q, depending on which direction the coefficients are read in:

Forward (increasing m): $Q_0 = P_0/a_j$, and for $m = 1, \ldots, B - 1$,

$$Q_m = \frac{P_m - (1 - a_j)\, Q_{m-1}}{a_j} = \frac{P_m}{a_j} - \frac{1 - a_j}{a_j}\, Q_{m-1}.$$

Backward (decreasing m): $Q_{B-1} = P_B/(1 - a_j)$, and for $m = B - 2, \ldots, 0$,

$$Q_m = \frac{P_{m+1} - a_j\, Q_{m+1}}{1 - a_j} = \frac{P_{m+1}}{1 - a_j} - \frac{a_j}{1 - a_j}\, Q_{m+1}.$$

Both compute the same exact quotient in exact arithmetic; they differ only in the coefficient c multiplying the running term each step - forward's $c_{\text{fwd}} = -(1 - a_j)/a_j$, backward's $c_{\text{bwd}} = -a_j/(1 - a_j)$ - which governs how a rounding error at step m is amplified by step $m + 1$.

Lemma 4.6 (Stability of bidirectional division)**.** *Choosing the forward recurrence when $a_j > 1/2$ and the backward recurrence when $a_j \leq 1/2$ gives per-step error amplification $|c| \leq 1$ for all $a_j \in (0, 1)$, with $|c| < 1$ strictly except at the boundary $a_j = 1/2$.*

Proof. $|c_{\text{fwd}}| = (1 - a_j)/a_j < 1 \iff a_j > 1/2$, and $|c_{\text{bwd}}| = a_j/(1 - a_j) \leq 1 \iff a_j \leq 1/2$; the two conditions are exhaustive and disjoint except at $a_j = 1/2$ itself, where both give $|c| = 1$. □

Lemma 4.6 proves $|c| \leq 1$ rigorously: when the stable branch is chosen, the per-step amplification factor never exceeds unity. However, a tight analytical bound on the *accumulated* rounding error as a function of B, connecting $|c|^B$ near the worst-case $a_j \approx 0.5$ to FP64's machine epsilon ($\approx 2.2 \times 10^{-16}$, roughly 15–16 significant decimal digits), was never derived in closed form. The division at each step introduces a factor of $1/a_j$ (or $1/(1 - a_j)$), which approaches 2 when $|c| \approx 1$, so the full error growth depends on more than $|c|^B$ alone.

Empirically, correctness was verified up to $B = 256$ on AMD Zen 4: bidirectional division with $B \in \{8, 16, 24, 32, 48, 64, 96, 128, 192, 256\}$ was tested against the V_1 exact reference across all `bench_grid verify` sizes and stack distributions, including the adversarial near-$a_j = 0.5$ regime the lemma flags as riskiest. All tests passed with no error growth across the entire range (standard: max 1.18×10^{-12} vs. 5×10^{-12} tolerance; adversarial: max 1.54×10^{-10} vs. 10^{-9} tolerance). Separately, an exhaustive block-size sweep across 95 (n, k) cells on the same hardware found that $B = 32$ is empirically the fastest choice regardless of k or n, and $B > 64$ is never faster (mean penalty 2.45× at $B = 256$). The production candidate list $\{8, 16, 24, 32, 48, 64\}$ therefore caps at 64 for performance reasons, not because of a hard correctness wall.

Rejected CPU optimizations. Several plausible optimizations were tried on the CPU engines and rejected after benchmarking. Karatsuba multiplication for the block build was 1.5× *slower* at every block size tested: on FMA-capable hardware, schoolbook's `c[i+j] += a[i]*b[j]` is already one FMA per term, so Karatsuba's trade of multiplies for additions saves nothing - both cost one cycle. FFTW's NEON codelets for FP64 were 3.4× slower than Apple's vDSP on Apple Silicon, which is why vDSP is dispatched for FFT execution instead of FFTW at supported sizes on that platform (Section 6); however, using vDSP directly inside the batched linear engine's inner loops (rather than only for FFT calls) was itself 2× *slower*, since per-call overhead at small k exceeds any vectorization benefit.

4.5 Engine Dispatch: Empirical Lookup Table

Engine dispatch must decide, for each (n, k) pair, whether to use the linear engine (memory-bound, $\mathcal{O}(nk)$) or the hybrid engine (compute-bound in the FFT phases, $\mathcal{O}(n \log^2 k)$). An earlier FMA-counting roofline model with harmonic-mean bandwidth blending correctly dispatched roughly 30/44 test cases on Zen 4: it had no knowledge of the cache hierarchy and could not distinguish L2-resident from DRAM-resident working sets. A later roofline model [Williams et al., 2009] improved this to 42/44 (95.5%) by modeling cache-level bandwidths explicitly, but the two remaining misses occurred where the engines were within 3–9% of each other, and individual constants in the formula were validated against real embedded execution only to find that the aggregate comparison still did not match the true measured crossover on either M3 Pro or Zen 4 hardware.

The replacement is an empirical crossover table: $N = 6$ calibrated n values with corresponding crossover k values where linear and hybrid engines have equal measured runtime, determined offline by binary search on real timing (median of 7 repetitions, $Q = 256$). At runtime, `select_engine`(n, k) performs log-linear interpolation between the two bracketing n values in the `crossover_n[]` table to obtain the crossover k for the requested n, then dispatches to linear if k is below the crossover and hybrid otherwise. This is the same structural precedent as LAPACK's `ILAENV ISPEC=3` (the `NX` parameter): a problem-size crossover between two algorithms, measured empirically per machine, consulted as a cheap runtime threshold with no live racing in production.

The block size B for the hybrid engine is selected by the same principle: `select_best_B`(n, k) performs a 2D nearest-neighbor lookup over a calibrated (n, k, B) grid (34 points per device), choosing the empirically fastest $B \in \{8, 16, 24, 32, 48, 64\}$. Unlike the crossover k, B is a discrete choice with no meaningful interpolation between adjacent values. Typical selections are $B = 32$ on both M3 Pro and Zen 4, with occasional $B = 24$ or $B = 48$ at specific (n, k) combinations.

Both tables (`crossover_n[]`/`crossover_k[]` and `bselect_n[]`/`bselect_k[]`/`bselect_B[]`) are generated offline by `tools/calibrate_crossover.c` and `tools/calibrate_best_b.c` respectively and stored in the per-device `fft_config.h`. This approach follows the broader autotuning tradition of measuring rather than modeling: FFTW's `PATIENT` planner times candidate FFT plans directly instead of relying on an indirect cost heuristic [Frigo and Johnson, 2005], and, for the specific mechanism of searching a parameter space empirically per architecture, is closest to the register-blocking search in the BeBOP/Sparsity system [Demmel et al., 2005], though that work searches register-blocking/unrolling parameters for one fixed small kernel size rather than building a size-indexed lookup table as done here.

Validation. The empirical crossover table produces correct dispatch decisions on all 42 bench_grid cells for M3 Pro and Zen 4 in serial mode, and on 83 of 84 cells across both platforms in parallel mode. The sole mismatch is Zen 4 parallel at $n = 65{,}536$, $k = n$, where the pure tree engine (T) edges out the hybrid engine (H) by a small margin; the dispatch model chooses between linear and hybrid only, and the tree engine's advantage in this regime is a parallel-specific effect (simpler clone structure with the same `PATIENT`-quality FFT plans) that the two-way dispatch does not attempt to capture.

5 Theoretical Analysis

Theorem 5.1 (Complexity of the tree engine)**.** *The tree engine (Algorithm 2) computes all n inner products in $\mathcal{O}(n \log^2 k)$ operations per quadrature point.*

Proof. Let $T_b(n)$ be the build cost for n leaves with truncation degree k. At the root, two subtrees of $n/2$ leaves are combined by truncated polynomial multiplication costing $\mathcal{O}(\min(n, k) \log \min(n, k))$ via FFT, giving $T_b(n) = 2T_b(n/2) + \mathcal{O}(\min(n, k) \log \min(n, k))$.

Case 1 ($n \leq k$): $T_b(n) = 2T_b(n/2) + \mathcal{O}(n \log n)$, solved by the Master theorem to $T_b(n) = \mathcal{O}(n \log^2 n)$.

Case 2 ($n > k$): all subproducts are capped at degree k, so merge cost is $\mathcal{O}(k \log k)$ regardless of subtree size. The recursion bottom is reached when subtree size $n/2^\ell \leq k$, i.e. at level $\ell = \log_2(n/k)$. Below this level, the tree has at most n/k independent subtrees of size $\leq k$, each costing $\mathcal{O}(k \log^2 k)$ by Case 1; a total of $(n/k) \cdot \mathcal{O}(k \log^2 k) = \mathcal{O}(n \log^2 k)$. Above saturation, there are $\log_2(n/k)$ levels each with $\leq n/(k \cdot 2^\ell)$ merges at $\mathcal{O}(k \log k)$ each, summing to $\sum_{\ell=1}^{\log_2(n/k)} \frac{n}{k \cdot 2^\ell} \cdot \mathcal{O}(k \log k) = \mathcal{O}(n \log k \cdot \sum_\ell 2^{-\ell}) = \mathcal{O}(n \log k)$. Adding the two contributions gives $\mathcal{O}(n \log^2 k + n \log k) = \mathcal{O}(n \log^2 k)$.

In both cases, $T_b(n) = \mathcal{O}(n \log^2 k)$. Propagation has the same complexity by adjoint parity (Remark after Theorem 4.4). □

Theorem 5.2 (Complexity of the hybrid engine)**.** *The hybrid engine with block size B computes all n inner products in $\mathcal{O}(nB + n \log^2 k)$ operations per quadrature point.*

Proof. Block build: n/B blocks at $\mathcal{O}(B^2)$ each $= \mathcal{O}(nB)$. Within-block divide: $\mathcal{O}(B)$ per player $= \mathcal{O}(nB)$.

For the inter-block tree, let $m = n/B$ leaves, each a degree-B polynomial. The tree has $\log_2 m$ merge levels above the leaves. Define the *saturation level* $s = \log_2(k/B)$: this is the number of levels where subtree degree remains below k, i.e. $B \cdot 2^\ell < k$ for $\ell \le s$ and capped at k thereafter.
Below saturation ($\ell = 1, \dots, s$). There are $m/2^\ell$ nodes at level ℓ, each merging two degree-$(B \cdot 2^{\ell-1})$ polynomials into one of degree $B \cdot 2^\ell$, costing $B \cdot 2^\ell \log_2(B \cdot 2^\ell)$ per merge. Per-level cost: $(m/2^\ell) \cdot B \cdot 2^\ell \log_2(B \cdot 2^\ell) = n \log_2(B \cdot 2^\ell)$. Summing over ℓ:

$$C_{\text{below}} = n \sum_{\ell=1}^{s} \log_2(B \cdot 2^\ell) = n \sum_{\ell=1}^{s} (\log_2 B + \ell) = n\Big(s \log_2 B + \frac{s(s+1)}{2}\Big).$$

Since $s = \log_2 k - \log_2 B$, the sum telescopes:

$$\begin{aligned} s \log_2 B + \frac{s(s+1)}{2} &= \frac{s}{2}\big(2 \log_2 B + s + 1\big) \\ &= \frac{s}{2}\big(\log_2 B + (\log_2 B + s) + 1\big) \\ &= \frac{s}{2}\big(\log_2 B + \log_2 k + 1\big) = \frac{s}{2} \log_2(2kB). \end{aligned}$$

Thus $C_{\text{below}} = \frac{n}{2}\, s \, \log_2(2kB)$.
At and above saturation ($\ell = s{+}1, \dots, \log_2 m$). All merges cost $k \log_2 k$ each. The number of nodes across these levels is a geometric series:

$$\sum_{\ell=s+1}^{\log_2 m} \frac{m}{2^\ell} = m\big(2^{-s} - 2^{-\log_2 m}\big) = \frac{n}{B}\Big(\frac{B}{k} - \frac{B}{n}\Big) = \frac{n}{k} - 1.$$

Hence $C_{\text{above}} = (n/k - 1) \cdot k \log_2 k = (n - k) \log_2 k$.

Total tree cost:

$$C_{\text{tree}}(n, B, k) = \frac{n}{2}\, \log_2\Big(\frac{k}{B}\Big)\, \log_2(2kB) \;+\; (n - k) \log_2 k. \tag{10}$$

At $B = k$ we have $s = 0$ and the first term vanishes, leaving $C_{\text{tree}} = (n - k) \log_2 k$, which is exactly the "all-saturated" cost of a tree whose leaves already meet the degree cap. At $B = 1$ we have $s = \log_2 k$, giving $C_{\text{tree}} = \frac{n}{2} \log_2 k \cdot \log_2(2k) + (n - k) \log_2 k = \frac{n}{2} \log_2^2 k + O(n \log k)$. For any $B \le k$, Equation 10 is $\Theta(n \log^2 k)$: the first term is $\Theta(n \log^2(k/B))$ and the second is $\Theta(n \log k)$; together they are $\Theta(n \log^2 k)$ because $\log(k/B) \le \log k$. □

Corollary 5.3 (Blocking does not improve the asymptotic order)**.** *The total hybrid cost $T(B) = nB + C_{tree}(n, B, k)$ is minimized at $B = \Theta(1)$, and increasing B beyond a small constant strictly increases the total.*

Proof. With Equation 10 and $s = \log_2(k/B)$, the change when doubling B is $\Delta T = T(2B) - T(B) = n(2B - B) + \frac{n}{2}\big[(s-1)\log_2(4kB) - s\log_2(2kB)\big]$. Substituting $\log_2(4kB) = \log_2(2kB) + 1$ and simplifying:

$$\begin{aligned}\Delta T &= nB + \frac{n}{2}\big[(s-1)(\log_2(2kB)+1) - s\log_2(2kB)\big] \\ &= nB + \frac{n}{2}\big[s - \log_2(2kB) - 1\big] \\ &= nB + \frac{n}{2}\big[(\log_2 k - \log_2 B) - (\log_2 k + \log_2 B + 1) - 1\big] \\ &= nB - n(\log_2 B + 1) = n\big(B - \log_2 B - 1\big).\end{aligned}$$

For $B = 1$: $\Delta T = n(1 - 0 - 1) = 0$, so $T(2) = T(1)$. For $B = 2$: $\Delta T = n(2 - 1 - 1) = 0$, so $T(4) = T(2)$. For $B \geq 4$: $f(B) = B - \log_2 B - 1$ has derivative $f'(B) = 1 - 1/(B \ln 2) > 0$ for $B > 1/\ln 2 \approx 1.44$, and $f(4) = 4 - 2 - 1 = 1 > 0$. Hence $\Delta T > 0$ for all $B \geq 4$, and $T(B)$ is strictly increasing on $B \in \{4, 8, 16, \ldots\}$.

Thus $T(1) = T(2) = T(4) < T(8) < T(16) < \cdots$, and the minimum is attained at $B \in \{1, 2, 4\}$. In particular, $B = \Theta(1)$. □

Theory versus practice. Corollary 5.3 shows that blocking provides no asymptotic speedup: the tree phase is $\Theta(n \log^2 k)$ regardless of B, and the nB block-build term grows faster than the tree-phase savings for any $B \geq 4$. The optimal B is therefore determined entirely by hardware constant factors: cache-line width, SIMD lane count, division latency; not by n or k. This is exactly what the empirical block-size sweep finds (Section 6): the wall-clock-optimal B on Zen 4 is $B = 32$, constant across the entire tested (n, k) grid, with no detectable dependence on k over the range $k \in [10^2, 10^5]$. Had blocking provided a $\Theta(\log k)$ asymptotic benefit, the optimal B would grow with k; the fact that it does not is an independent experimental confirmation of Corollary 5.3.

Numerical stability of the hybrid engine's within-block division is established separately, alongside the division recurrence itself, in Section 4.4 (Lemma 4.6).

6 CPU Experimental Results

6.1 Experimental Setup

I evaluate the CPU implementation on two platforms (GPU hardware and setup are described separately in Section 8). All experiments use $Q = 256$ quadrature points, uniform chip stacks, and report the median of 5 repetitions. Correctness is verified against closed-form V_1 and V_2 equities (see Appendix) and cross-checked between engines.

- **Apple M3 Pro:** 6 performance + 6 efficiency cores (12 logical), ARM64 with NEON (2 FP64/vector), 4 FMA ports per core. FFT: FFTW3 PATIENT wisdom calibrated on this specific M3 Pro unit, with Apple vDSP dispatch at 33 supported sizes.
- **AMD Ryzen 9 7950X (Zen 4):** 16 homogeneous cores, AVX-512 (8 FP64/vector), 2 FMA units per core, 1 MB per-core L2, 32 MB shared L3. FFT: AOCL-FFTW, znver4-tuned, sole backend (no dual-library dispatch; see Related Work). Memory on this machine runs at 3600 MT/s rather than its DIMMs' 5600 MT/s rating, an AM5 two-DIMMs-per-channel electrical limit rather than a configuration fault. Its effect on these results is not a flat scaling factor: the compute-bound linear engine is essentially unaffected, while the FFT-heavy hybrid engine is memory-bandwidth-bound and correspondingly slower at large k. The Zen 4 hybrid

Table 1: Single-threaded execution time (ms), $Q = 256$, uniform stacks. Best engine per cell: L = linear, H = hybrid, T = tree.

	Apple M3 Pro						**AMD Zen 4 (AOCL-FFTW)**					
n	k=10	k=50	k=100	k=n/4	k=n/2	k=n	k=10	k=50	k=100	k=n/4	k=n/2	k=n
1024	1.79(L)	7.28(L)	12.8(L)	18.4(H)	22.2(H)	24.0(H)	1.46(L)	3.48(L)	7.32(L)	19.1(H)	22.3(H)	24.1(H)
2048	4.01(L)	14.1(L)	26.2(L)	47.9(H)	54.8(H)	57.3(H)	3.67(L)	8.21(L)	15.0(L)	48.4(H)	53.7(H)	57.2(H)
4096	8.10(L)	28.2(L)	52.4(L)	115(H)	135(H)	155(T)	7.39(L)	17.8(L)	25.8(L)	115(H)	128(H)	139(H)
8192	16.1(L)	56.4(L)	104(L)	273(H)	325(H)	359(T)	13.4(L)	34.3(L)	53.6(L)	277(H)	315(H)	341(H)
16384	32.3(L)	113(L)	210(L)	650(H)	725(H)	778(H)	29.6(L)	56.9(L)	117(L)	667(H)	743(H)	810(H)
32768	65.0(L)	227(L)	420(L)	1590(H)	1800(H)	1970(T)	62.3(L)	131(L)	218(L)	1570(H)	1790(H)	1950(H)
65536	131(L)	456(L)	842(L)	3460(H)	3880(H)	4120(H)	118(L)	266(L)	411(L)	4110(H)	4750(H)	5110(H)

Table 2: Parallel execution time (ms), $Q = 256$, uniform stacks. M3 Pro: 12 threads (6P+6E). Zen 4: 16 threads. Best engine per cell: L = linear, H = hybrid, T = tree.

	Apple M3 Pro (12 threads)						**AMD Zen 4 (16 threads)**					
n	k=10	k=50	k=100	k=n/4	k=n/2	k=n	k=10	k=50	k=100	k=n/4	k=n/2	k=n
1024	0.329(L)	1.23(L)	2.19(H)	2.64(H)	2.99(H)	3.27(H)	0.121(L)	0.359(L)	1.21(L)	1.41(H)	1.66(H)	1.78(H)
4096	1.36(L)	4.79(L)	8.11(H)	16.1(H)	18.5(H)	21.5(H)	0.651(L)	1.48(L)	2.35(L)	8.40(H)	9.36(H)	10.1(H)
8192	2.67(L)	9.44(L)	16.4(H)	37.9(H)	46.9(H)	50.4(T)	1.23(L)	2.84(L)	4.63(L)	20.4(H)	24.2(H)	28.1(H)
16384	5.40(L)	18.7(L)	35.1(L)	91.9(H)	101(H)	111(H)	2.37(L)	6.43(L)	9.69(L)	88.1(H)	112(H)	142(H)
32768	10.7(L)	37.9(L)	67.7(H)	229(H)	265(H)	296(T)	6.04(L)	13.6(L)	23.6(L)	280(H)	314(H)	363(H)
65536	21.6(L)	77.4(L)	142(L)	515(H)	583(H)	649(H)	21.0(L)	30.4(L)	48.4(L)	627(H)	729(H)	901(H)

columns below are therefore a bandwidth-limited lower bound for that microarchitecture, not its ceiling.

6.2 CPU Performance

Tables 1 and 2 report execution times across a grid of (n, k) pairs. All numbers are from real measurements on the target hardware (`bench_grid`; M3 Pro July 24, 2026, Zen 4 July 27, 2026); none are estimated or borrowed. The best engine per cell is shown, with dispatch decisions made automatically by the empirical crossover table.

Platform comparison. Zen 4 benefits from AVX-512's 8-wide FP64 FMA (versus NEON's 2-wide) in the linear engine, and from AOCL-FFTW's znver4-tuned AVX-512 codelets in the hybrid engine.

A direct A/B comparison of AOCL-FFTW versus the upstream (plain) FFTW distribution was conducted on the same Zen 4 machine, at the same FFT size (d = 32,768), with the same `FFTW_PATIENT` flag, under the `performance` cpufreq governor. AOCL-FFTW completed the per-pipeline FFT work in 53–57 μs versus 66–69 μs for plain FFTW; a 20–25% advantage, reproducible across repeated runs, with no calibrated size where the reverse held. AOCL-FFTW is therefore used as the sole FFT backend on Zen 4; no dual-library dispatch is needed on this platform (contrast the GPU's cuFFT/cuFFTDx tier selection, Section 8, where dispatch genuinely depends on problem size).

M3 Pro benefits from Apple vDSP's interleaved DFT dispatch at 33 supported FFT sizes (10–18% faster than FFTW at those sizes) and from the integration of NEON SIMD throughout the linear engine's batched path ($B_Q = 8$ quadrature points interleaved).

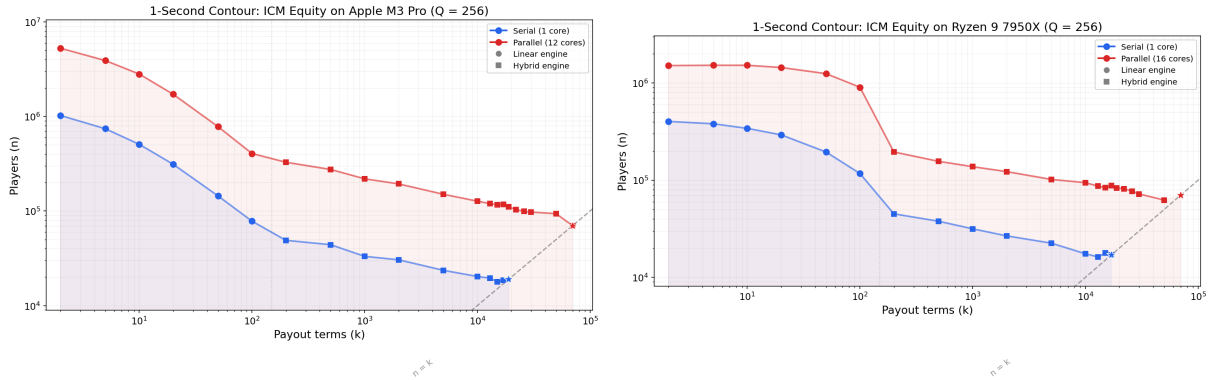

Figure 1: One-second contour: maximum n achievable within 1 s as a function of k. Left: Apple M3 Pro. Right: AMD Zen 4. Solid line: serial; dashed line: parallel. The cliff between $k = 100$ and $k = 200$ (Zen 4) marks the linear-to-hybrid engine transition.

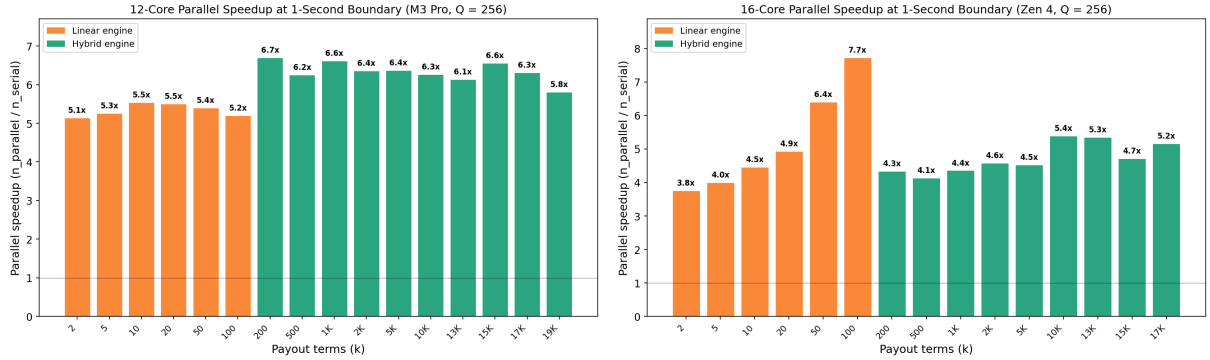

Figure 2: Parallel speedup at the one-second boundary. Left: M3 Pro (12 threads, 6P+6E). Right: Zen 4 (16 threads).

Parallel scaling asymmetry. The two engines exhibit markedly different parallel scaling behavior. The linear engine achieves 7–8× speedup on 16 Zen 4 cores, while the hybrid engine achieves only 3.6–4.2×. The explanation is memory hierarchy: the linear engine's L2-aware checkpointing ensures each thread's working set fits in its private per-core L2 cache (aggregate $16 \times 119 \approx 1{,}900$ GB/s), while the hybrid engine's FFT-dominated tree traversal operates on polynomials far exceeding L2 capacity, forcing all 16 threads to compete for the shared DRAM bus.

This asymmetry creates a sharp cliff at the linear-to-hybrid transition, which on Zen 4 falls between $k = 100$ and $k = 200$: the parallel linear engine at $k = 100$ handles 906,259 players within one second, but the parallel hybrid engine at $k = 200$ handles only 195,356 - a 4.6× drop that reflects the bandwidth regime change, not an algorithmic deficiency. The serial contour shows the same transition at the same k and in the same direction (117,264 at $k = 100$ falling to 45,085 at $k = 200$), but a shallower 2.6× drop, since a single thread does not saturate the DRAM bus.

6.3 Quadrature Accuracy

The erfc-trap quadrature rule - trapezoidal rule after the change of variables $v = \Phi(y)$ - achieves exponential convergence for smooth integrands [Trefethen and Weideman, 2014]. I compare it against tanh-sinh (double-exponential) quadrature [Mori and Sugihara, 2001] across four stack

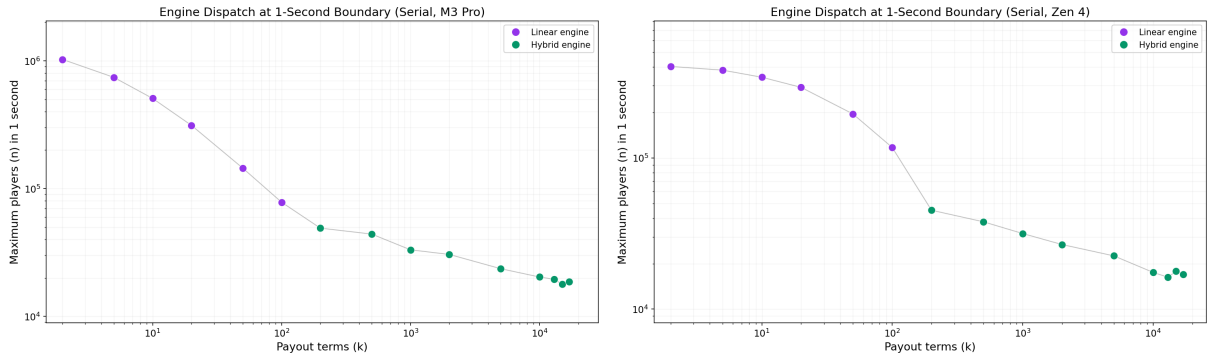

Figure 3: Engine dispatch decisions at the one-second boundary. Purple: linear engine. Green: hybrid engine. Left: M3 Pro. Right: Zen 4.

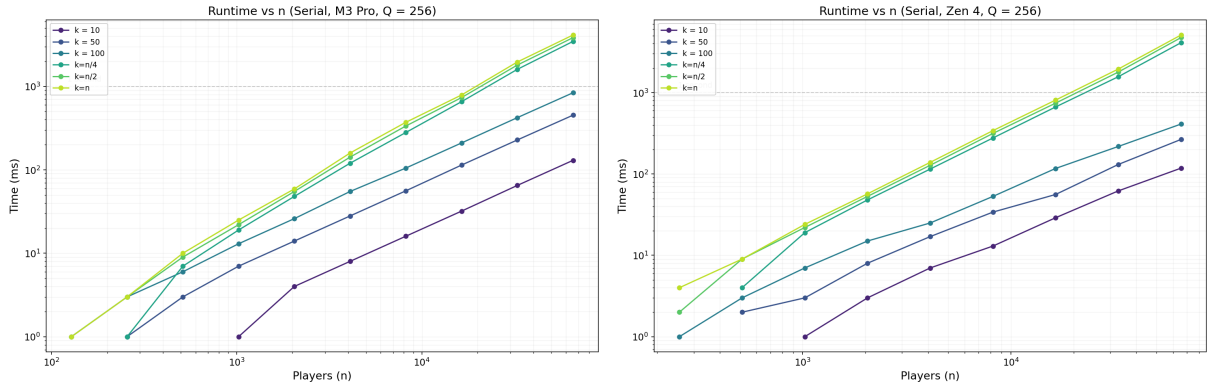

Figure 4: Runtime vs. n at fixed k, single-threaded. Left: Apple M3 Pro. Right: AMD Zen 4. The CPU analogue of the GPU runtime plot in Figure 7.

distributions:

- **Uniform:** $S_i = 100$ for all i (ratio 1).
- **Adversarial:** $S_1 = 10{,}000$, $S_i = 1$ for $i > 1$ (ratio 10^4).
- **Geometric:** $S_i = 2^{i-1}$, giving $S_{\max}/S_{\min} \approx 5 \times 10^5$ at $n = 20$.
- **Extreme adversarial:** $S_1 = 10^9$, $S_i = 1$ for $i > 1$ (ratio 10^9). This is the practical worst case.

For each configuration I measure the maximum relative error across all n players against closed-form V_2 equities. Table 3 reports results for $n = 20$, the hardest case.

Three findings emerge. First, erfc-trap converges to $\sim 10^{-12}$ relative error by $Q = 128$ on uniform and adversarial stacks, and by $Q = 256$ on the more demanding geometric and extreme distributions. At the $Q = 256$ production default, all distributions achieve relative error below 1.6×10^{-10}.

Second, tanh-sinh achieves a lower error floor on uniform distributions ($\sim 10^{-14}$ at $Q = 256$) but degrades catastrophically at extreme ratios, reaching only 6.5×10^{-6} on the 10^9 extreme case and failing to improve further at $Q = 1024$. The double-exponential substitution's strip of analyticity narrows as $S_{\max}/S_{\min}$ grows; beyond some threshold, convergence fails entirely.

Table 3: Maximum relative error vs. quadrature order Q for $n = 20$. Gauss = erfc-trap; TS = tanh-sinh. Machine epsilon (FP64) is 2.2×10^{-16}.

	Uniform		Adversarial		Geometric		Extreme (10^9)	
Q	Gauss	TS	Gauss	TS	Gauss	TS	Gauss	TS
4	3.9	2.5×10^{-1}	1.0×10^{0}	1.2×10^{1}	5.0	1.1×10^{1}	1.0×10^{0}	1.0×10^{0}
8	6.2×10^{-3}	1.0	2.3×10^{0}	9.8×10^{-1}	2.4	1.0×10^{0}	9.9×10^{-1}	1.0×10^{0}
16	1.6×10^{-1}	6.6×10^{-1}	1.4×10^{-2}	1.6×10^{0}	7.7×10^{-1}	1.4×10^{0}	1.2×10^{0}	4.1×10^{0}
32	4.3×10^{-3}	8.5×10^{-4}	5.2×10^{-2}	3.0×10^{-1}	9.9×10^{-2}	6.4×10^{-1}	2.8×10^{-1}	2.7×10^{-1}
64	3.0×10^{-7}	1.5×10^{-4}	1.8×10^{-4}	1.4×10^{-2}	1.5×10^{-3}	4.9×10^{-2}	9.3×10^{-3}	2.1×10^{-1}
128	1.3×10^{-12}	1.5×10^{-11}	5.3×10^{-10}	3.9×10^{-6}	2.3×10^{-7}	6.5×10^{-4}	4.7×10^{-5}	2.4×10^{-2}
256	1.7×10^{-12}	3.6×10^{-14}	5.0×10^{-13}	1.4×10^{-12}	5.0×10^{-13}	1.5×10^{-8}	1.5×10^{-10}	6.5×10^{-6}
512	1.9×10^{-12}	2.5×10^{-14}	5.8×10^{-13}	7.2×10^{-13}	5.9×10^{-13}	4.2×10^{-11}	4.9×10^{-13}	5.9×10^{-8}

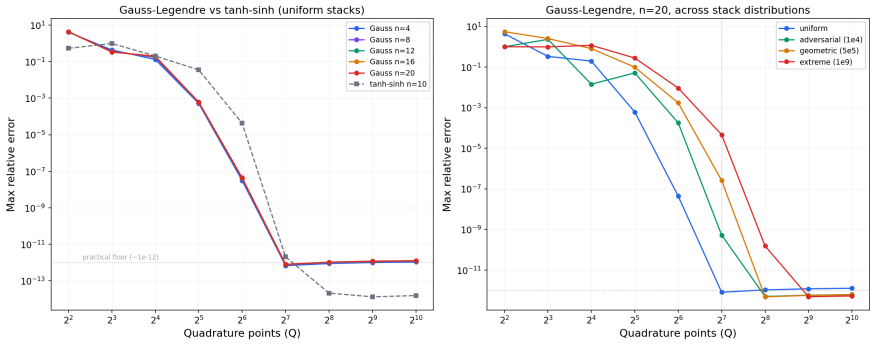

Figure 5: Convergence of erfc-trap (Gauss-Legendre) quadrature toward machine precision as Q increases, across the four stack distributions of Table 3.

Third, this failure mode drove the choice of quadrature scheme. In the typical case, tanh-sinh is the better method. But a general-purpose ICM library must handle arbitrary stack distributions, including cases where one player holds 10^9 times as many chips as another. Gaussian quadrature's uniform reliability - it keeps converging, predictably, all the way to the 10^9 bound - was judged more valuable than better average-case precision.

6.4 Calibration Constants

Table 4 summarizes the hardware constants measured via offline profiling. All bandwidth values are from streaming benchmarks on the target hardware, not copied from vendor specifications. τ_{FMA} feeds the linear engine's $\mathcal{O}(nk)$ compute-bound cost estimate (Section 4.1); W_{L2}, W_{L3}, and W_{DRAM} feed the linear engine's L2-aware checkpointing logic and the (now-superseded) roofline dispatch model described in Section 4.5.

Note: M3 Pro's shared L2 architecture (16–36 MB total, configuration-dependent) does not fit neatly into the per-core L2/L3 model, so only the 1 MB performance-core L2 is listed. The bandwidth values reflect measured streaming throughput at the respective cache levels. † Zen 4's bandwidth-constant measurement pipeline produced values inconsistent with plausible hardware limits (e.g. a raw-file reading in the hundreds-of-TB/s range for W_{L2}), a pre-existing bug traced to `tools/calibrate.c`'s streaming microbenchmark: its inner loop body does not depend on the outer repetition counter, so an optimizing compiler can prove the repeated stores are redundant

Table 4: Platform calibration constants.

Constant	M3 Pro	Zen 4	Description
τ_{FMA} (ns)	0.050	0.079	Scalar FMA latency
W_{L2} (GB/s)	113.3	112†	Measured L2 streaming BW
W_{L3} (GB/s)	111.3	34†	Measured L3 streaming BW
W_{DRAM} (GB/s)	110.7	32†	Measured DRAM streaming BW
L_2 (bytes)	1 MB	1 MB	Per-core L2 size
L_3 (bytes)	-	32 MB	Shared L3 size
Typical B	16	32	Cost-model block size
FFT backend	FFTW+vDSP	AOCL (sole)	Library dispatch

and collapse the whole repetition loop to a single real pass, while the byte-count computation still charges for every nominal repetition, inflating the reported bandwidth by approximately the repetition factor. Dividing each Zen 4 value by its own repetition count gives 112, 34, and 32 GB/s for W_{L2}, W_{L3}, and W_{DRAM} respectively; all physically plausible; consistent with this exact mechanism. The same source does not exhibit the bug when compiled for M3 Pro (its values were already sane), consistent with a GCC/x86 optimization difference rather than a flaw in the benchmark's logic. Fixed with a standard compiler barrier after each repetition; verified not to regress M3 Pro's already-correct values. **Not yet re-verified with a fresh calibration run on Zen 4 hardware**; the calibration machine became unreachable before the fix was written, so the corrected values above are a well-evidenced prediction, not a fresh measurement. These constants feed the linear engine's L2-aware checkpointing cost, not correctness; `bench_grid verify` is unaffected regardless.

6.5 Calibration Methodology: Direct Measurement vs. Indirect Regression

The empirical crossover table (Section 4.5) replaced an earlier roofline cost model that depended on hardware constants: τ_{FMA}, cache bandwidths, per-size FFT costs, and several micro-architectural parameters; that are fit to timing data by nonlinear least squares. Two of these constants proved structurally unrecoverable from aggregate plan-timing data alone, leading to a methodological shift from indirect regression to isolated direct microbenchmarking. This section describes the failure mode, the fix, and the precedent it follows.

Precedent: direct measurement over indirect inference. The principle of measuring a parameter directly rather than backing it out of aggregate data has strong precedent in high-performance computing. FFTW's `PATIENT`-mode planner measures the actual execution time of candidate FFT plans directly rather than relying on an indirect cost model [Frigo and Johnson, 2005]. The BeBOP/Sparsity system's register-blocking search empirically times generated kernel variants per architecture to select the fastest [Demmel et al., 2005]. The roofline model measures peak FLOP/s and memory bandwidth via dedicated streaming microbenchmarks rather than inferring them from application runs [Williams et al., 2009]. In each case, the rationale is the same: when a parameter's contribution to the total observed cost is small relative to other, larger effects, an indirect regression lacks the signal to identify it reliably.

Persistency of excitation. This failure mode has a formal name in control theory and system identification: *persistency of excitation* [Ljung, 1999]. A parameter is identifiable from training

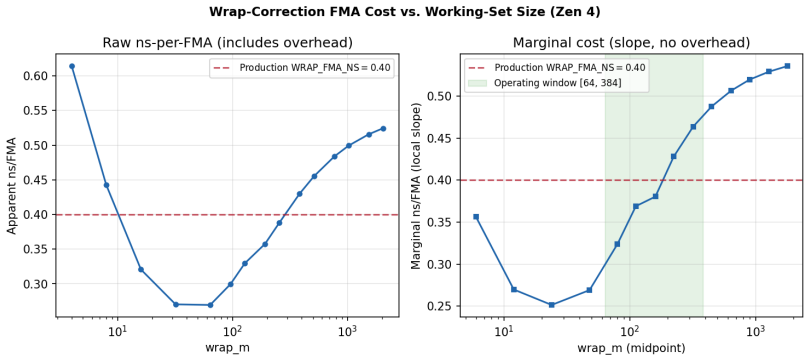

Figure 6: Measured marginal cost of the wrap-correction FMA loop as a function of wrap range, for the SMALL_2048 polynomial-size regime (Zen 4). The curvature reflects a real cache-hierarchy transition: L1/L2-resident at small `wrap_m` ($\sim$0.26 ns/FMA), transitioning to memory-latency-bound at large `wrap_m` ($\sim$0.53 ns/FMA). The production value `WRAP_FMA_NS` $= 0.40$ (dashed line) is a representative point estimate from the least-squares slope over the `wrap_m` $\in [64, 384]$ operating window.

data only if the input signal varies its effect enough to be distinguishable from noise and from the effects of other parameters. When one parameter's contribution never exceeds a few percent of any training example's total cost (as is the case for wrap-correction cost in the tree engine; see below), the regression objective is effectively flat with respect to that parameter: wildly different values produce nearly identical aggregate predictions, and the fitted value reflects optimizer drift rather than physical reality.

The wrap-correction identifiability failure. The tree engine's wrap correction (Algorithm 2) corrects aliasing wraparound when truncated polynomial multiplication is performed via FFT with a transform size smaller than the full linear convolution length. The cost of this correction, parameterized by `WRAP_FMA_NS` (nanoseconds per fused multiply-add in the correction loop), is a small fraction of any sampled plan's total predicted time. On AMD Zen 4, only 5 of 200 sampled plans had wrap-correction cost exceeding 1% of that plan's total predicted time (maximum 1.5%). The nonlinear least-squares fit's objective was flat across a factor of 4 in `WRAP_FMA_NS` ($\Delta < 0.001$ in the objective), making the constant unidentifiable from aggregate data.

The fix was to measure `WRAP_FMA_NS` directly: an isolated microbenchmark (`tools/bench_wrap_fma.c`) that executes only the wrap-correction inner loop (a verbatim copy of the production code in `src/icm.c`), sweeping the wrap range over $[4, 2048]$ across three polynomial-size regimes (2K, 8K, 32K elements) so that the correction dominates the measured time by construction ($R^2 = 0.9998$). The result, displayed in Figure 6, is not a single constant but a genuine cache-hierarchy-transition curve: the marginal cost rises smoothly from $\sim$0.26 ns/FMA at small wrap ranges (L1/L2-resident, near-FMA-throughput-bound) to $\sim$0.53 ns/FMA at large wrap ranges (memory-latency-bound as the working set exceeds cache capacity). At the decision-relevant operating range (`wrap_m` $\approx$ 64–384, matching tree levels 7–9 for typical (n, k) pairs), the local slope is $\sim$0.36–0.45 ns/FMA.

Disclosure: point-estimate approximation. The shipped `WRAP_FMA_NS` values, 0.40 on Zen 4 and 0.4942 on Apple M3 Pro, are representative point estimates from a hand-chosen operating-range window (`wrap_m` $\in [64, 384]$), not a fully piecewise or continuous model of the cache-hierarchy transition visible in Figure 6. The dispatch model treats wrap correction as having a single constant marginal cost; a more accurate model would account for the dependence on working-set size, but

Table 5: Speedup of subset equity vs. full-equity computation, M3 Pro, single-threaded, $Q = 256$. Ratios < 1 indicate net slowdown.

n	Speedup @1% targets	Speedup @5–10% targets	Speedup @≥25% targets
1024	0.92	0.91–0.92	0.91
4096	1.39	1.09–1.12	1.05–1.07
16384	1.27	1.06–1.08	1.03–1.05
65536	1.16	1.04–1.05	1.02–1.03

the single-constant approximation has proved sufficient in practice (dispatch accuracy 42/44 on Zen 4, Section 4.5).

M3 Pro: a second identifiability failure. The same methodology was applied to `FP64_DIV_NS` on the Apple M3 Pro after the indirect fit converged to a physically implausible value (0.5 ns, at its imposed lower bound; FP64 division latency on Apple Silicon is expected to be 13–16 cycles, or ∼3.2–4.0 ns at 4.05 GHz). A second dedicated microbenchmark (`tools/bench_div_chain.c`) measured the dependency-chained division cost directly (matching the actual usage in the hybrid engine's leaf-extraction synthetic-division recurrence, Section 4.4), yielding 3.4890 ns.

Collinearity caveat (disclosed, not hidden). Pinning both `WRAP_FMA_NS` and `FP64_DIV_NS` to their directly-measured values on M3 Pro raised the aggregate fit's RMS log-relative error from 6.57% to 10.2% and pushed `FFT_OVERHEAD_NS` to a physically odd 631 ns (and τ_{FMA} to its fit lower bound) to compensate. The current `sample_plans` training data does not cleanly separate the division constant's effect from the schoolbook-multiplication constant's on this hardware; a collinearity limitation in the training dataset, not a correctness issue. The dispatch decisions remain sound: `bench_grid verify` passes all tests and `bench_grid crossover` shows a clean, monotonic linear-to-hybrid transition at $k \approx 250$–260. Improving the `sample_plans` coverage to break this collinearity is open follow-up work.

6.6 Subset Equity Pruning

The hybrid engine supports a subset-query mode that computes equities for only a specified subset of target players. It exploits *target-locality pruning*: a per-block hot/cold bitmask is propagated bottom-up through the tree, and cold branches are skipped during the top-down propagation phase. This optimization is correct (verified bit-exact against the full-equity engine) but has a fixed upfront cost for mask construction and per-node branch checks.

Table 5 reports speedups measured on M3 Pro. The pruning delivers a net win only for $n \geq 4096$ at low target fractions ($\leq$ 5–10%), topping out at ∼1.4× speedup at $n = 4096$ with 1% targets, and fading to breakeven by 25% targets. At $n = 1024$, it is a net loss at every target ratio tested.

7 GPU Implementation: NVIDIA B200

The GPU implementation targets the NVIDIA B200 and adapts the hybrid block-tree algorithm to the GPU execution model. The linear engine is not applicable: its sequential player-by-player structure cannot saturate GPU parallelism. Only the tree-based engines, where all nodes at each level are independent, map naturally to GPU execution.

7.1 Architecture

The tree's level-by-level structure becomes a sequence of batched kernel launches. At the bottom levels of a tree with $n = 1{,}572{,}864$ leaves and $Q = 256$, this yields millions of independent operations - far more than needed to saturate the GPU's streaming multiprocessors.

7.2 Three-Tier Kernel Dispatch

The planner assigns each subproduct-tree level to one of three kernel tiers based on polynomial degree:

- **Schoolbook** ($d < 64$): shared-memory kernels with excellent coalescing and zero launch overhead.
- **cuFFTDx fused** ($64 \leq d \leq 4096$): device-side FFT kernels that fuse forward R2C, pointwise multiply, and inverse C2R into a single on-device launch, eliminating intermediate global-memory round-trips.
- **Batched cuFFT** ($d > 4096$): globally-optimized plans for the highest-throughput regime.

The entire multi-level execution sequence, spanning dozens of kernel launches across all three tiers - is captured into a single CUDA Graph and replayed with $\sim$2.5 μs total launch overhead.

7.3 Key Optimizations

FFT-stride layout. The single largest optimization: polynomials are stored at their FFT-padded stride rather than at their logical degree. This eliminates the gather/scatter passes that cuFFT's batched interface would otherwise require, cutting two full memory passes per FFT call.

Arena allocator. The baseline implementation made 35 separate `cudaMalloc` calls. Replacing these with a single arena allocation eliminates serialization at the CUDA driver layer.

Q-batch pipelining. Rather than processing all $Q = 256$ quadrature points in one monolithic batch, I process $Q_{\text{batch}} = 4$–8 points at a time, overlapping the block-build phase of one batch with tree propagation of the previous batch using multi-stream execution.

Rejected optimizations. Two approaches were tested and rejected after benchmarking:

- **cuFFT callbacks:** Fusing the pointwise multiply into cuFFT's load/store callbacks was 2.3× *slower* than the separate-kernel approach; per-element callback overhead breaks memory coalescing.
- $B = 1$ **balanced tree:** Using single-player leaves (no block structure) was 20–56% slower because the tree performs 70% more FMA operations and adds kernel launch barriers.

8 GPU Experimental Results

Hardware. NVIDIA B200, sm_100, $\sim$180 GB HBM3e. All experiments use $Q = 256$ quadrature points and cuFFTDx fused device-side kernels with CUDA graph capture (Section 7).

Table 6 reports a systematic (n, k) grid sampled directly from the same 211-point calibration heatmap that generates Figure 7, in the same row/column layout as Tables 1–2 so the CPU and

Table 6: NVIDIA B200 GPU execution time (ms), $Q = 256$, FP64. Systematic (n, k) grid sampled from the calibration heatmap.

n	k=64	k=1024	k=$n/2$	k=n
4,096	0.35	0.74	0.82	0.84
16,384	1.16	2.79	3.85	4.10
65,536	4.35	10.94	19.36	20.22
262,144	17.09	43.37	96.31	100.49
1,048,576	77.45	187.72	508.24	513.41
4,194,304	272.98	771.48	2,383.66	2,340.89
33,554,432	2,639.85	6,141.28	22,584.75	23,116.73

Table 7: NVIDIA B200 one-second threshold, located by binary search on measured time rather than sampled from the grid above. Each pair brackets the 1000 ms crossing; the search terminated at a bracket width of 15,360 on both curves.

Curve	n	Time (ms)
$k = n$	1,490,944	918.8
	1,506,304	1,124.2
$k = 100$	7,975,936	998.9
	7,991,296	1,001.1

GPU results are directly comparable. Table 7 separately reports the one-second threshold located by a dedicated binary search on measured time (`tools/threshold_search_gpu.cu`, median of 5 repetitions per candidate n) rather than by interpolating the systematic grid - these are the specific n values that pin down the 1-second boundaries quoted below, and are not a subsample of Table 6.

At $k = n$, the GPU computes all 262,144 player equities in 100.5 ms. The under-one-second frontier for $k = n$ is $n = 1{,}490{,}944$ (918.8 ms); the next candidate the search probed above it, $n = 1{,}506{,}304$, crosses to 1,124.2 ms. At $k = 100$ the frontier reaches $n = 7{,}975{,}936$ players (998.9 ms). The frontier is limited by VRAM at large k and by tree-level cliffs at large n.

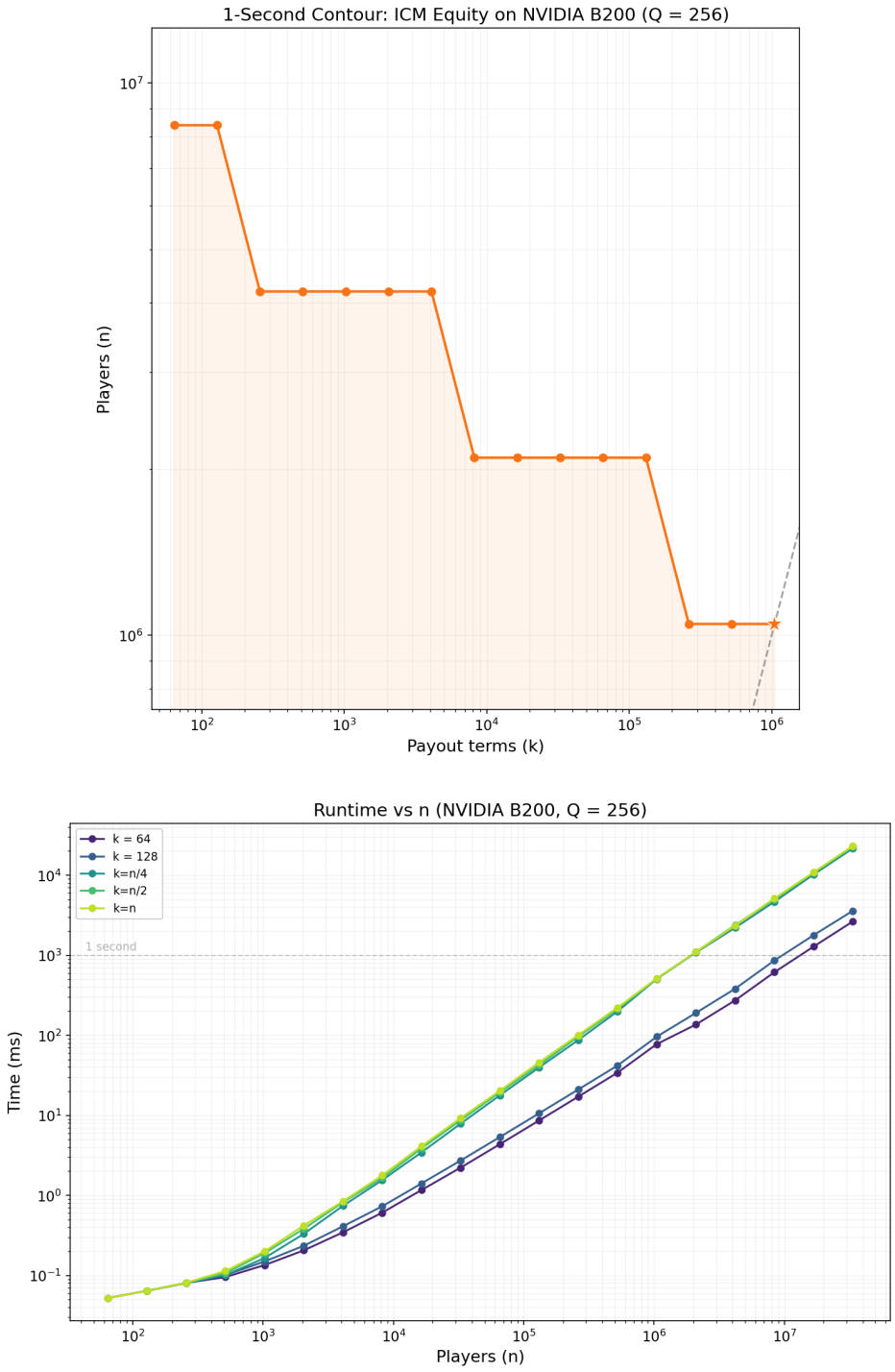

Figure 7: NVIDIA B200. Top: one-second contour (maximum n achievable within 1 s as a function of k), the GPU analogue of Figure 1. Bottom: runtime vs. n at fixed k, showing the tree-level cliffs referenced below.

9 Conclusion

I have presented a high-performance algorithm for computing ICM tournament equities using generating-function quadrature and FFT-accelerated subproduct trees. The algorithm achieves $\mathcal{O}(Qn \log^2 k)$ time, double-precision accuracy via $Q = 256$ quadrature points (relative error $< 5 \times 10^{-12}$ on typical inputs), and automatically dispatches between a linear engine (optimal for small k) and a hybrid block-tree engine (optimal for large k) using an empirically calibrated crossover table (Section 4.5).

The propagation phase is precisely the reverse-mode derivative of the build phase, and recognizing this gives both a clean correctness proof (Theorem 4.4) and complexity parity between build and propagation for free. The dominant cost throughout is data movement, not arithmetic: the empirical dispatch table, L2-aware checkpointing, and the GPU's FFT-stride layout all exist because moving data between cache levels is the binding constraint, not the FMA count. And the cost model and FFT dispatch decisions throughout - per-size FFT timing, per-level cache bandwidths, dual-library selection - are driven by direct calibration against measured hardware behavior rather than predicted from first-principles constants, because measuring turned out to be cheaper and more reliable than modeling it a priori.

On the CPU, the batched linear and hybrid engines compute exact equities for up to 17,984 players in one second, single-threaded, on AMD Zen 4. A CUDA implementation for NVIDIA B200 extends this to roughly 1.5 million players in about a second, enabling ICM analysis at field sizes that were previously intractable.

Future work includes exploring real-time tournament decision support where subset equity queries can exploit the active-player masking already present in the implementation. A more speculative direction: since the propagation phase is already reverse-mode automatic differentiation of the build phase (Section 4.2), and $a_j(v) = v^{S_j}$ is itself differentiable in S_j, extending this AD lens one step further - differentiating through the quadrature integral itself - would give exact gradients $\partial E_i / \partial S_j$ at the same $\mathcal{O}(Qn \log^2 k)$ cost as the equities themselves, potentially useful as a differentiable ICM node in gradient-based tournament-strategy training objectives.

A Closed-Form Reference Equities

The library is validated against exact closed-form reference values for two special payout structures. Both follow from a single combinatorial identity applied at different orders, combined with linearity of expectation - not from enumerating elimination orderings - and are exact for any n.

Write player i's finishing position as $m = M + 1$, where M is the (random) number of other players who finish ahead of i ($M = 0$ is first place). For any $t \geq 0$, the number of ways to choose t of the $n - 1 - M$ players who finish *behind* i satisfies, on every realized outcome,

$$\binom{n-1-M}{t} = \sum_{\substack{T \subseteq [n] \setminus \{i\} \\ |T| = t}} \mathbf{1}[i \text{ finishes better than every player in } T], \tag{11}$$

since this is exactly choosing t of the $n-1-M$ players ranked below i - no probability is involved yet, it is a counting identity on one outcome. Taking expectations and applying linearity term-by-term to the sum of indicators on the right (valid regardless of how the indicator events are correlated

with each other):

$$E\left[\binom{n-1-M}{t}\right] = \sum_{\substack{T\subseteq[n]\setminus\{i\}\\|T|=t}} \Pr[i \text{ beats every player in } T]. \tag{12}$$

"i beats every player in T" means i's exponential clock is the smallest among $\{i\}\cup T$; by the same competing-exponentials argument as Section 3 (the minimum of independent $\mathrm{Exp}(S_j)$ variables is itself exponential with rate equal to the sum of the rates), $\Pr[i \text{ beats every player in } T] = S_i/(S_i + \sum_{j\in T} S_j)$.

V1 (linear payout, $\pi_m = n - m + 1$). Since $n - M = \binom{n-1-M}{0} + \binom{n-1-M}{1}$, apply Equation 12 at $t = 0$ (trivially 1, the empty subset $T = \emptyset$) and $t = 1$ (one term per opponent j, $T = \{j\}$):

$$\begin{aligned} E_i = E[n-M] &= E\left[\binom{n-1-M}{0}\right] + E\left[\binom{n-1-M}{1}\right] \\ &= 1 + \sum_{j\neq i} \Pr[i \text{ beats every player in } \{j\}] \\ &= 1 + \sum_{j\neq i} \frac{S_i}{S_i + S_j}. \end{aligned}$$

V2 (quadratic payout, $\pi_m = \binom{n-m}{2}$). Since $\pi_m = \binom{n-m}{2} = \binom{n-1-M}{2}$ directly, apply Equation 12 at $t = 2$ (one term per opponent pair $T = \{j, k\}$):

$$\begin{aligned} E_i = E\left[\binom{n-1-M}{2}\right] &= \sum_{j<k,\, j,k\neq i} \Pr[i \text{ beats every player in } \{j,k\}] \\ &= \sum_{j<k,\, j,k\neq i} \frac{S_i}{S_i + S_j + S_k}. \end{aligned}$$

Higher payout schedules follow the same pattern for larger t; V1 and V2 are the $t \leq 2$ cases used here as exact, closed-form, arbitrary-n ground truth. Both are implemented as `icm_v1_exact()` and `icm_v2_exact()` in the reference library and used to validate the quadrature-based implementation across all stack distributions.

References

Advanced Micro Devices, Inc. AOCL-FFTW: AMD optimized CPU libraries — FFTW. `https://www.amd.com/en/developer/aocl/fftw.html`, 2024. 636 AVX-512 codelets for AMD Zen architectures.

James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. doi: 10.1090/S0025-5718-1965-0178586-1.

James Demmel, Jack Dongarra, Victor Eijkhout, Erika Fuentes, Antoine Petitet, Rich Vuduc, R. Clint Whaley, and Katherine Yelick. Self-adapting linear algebra algorithms and software. In *Proceedings of the IEEE*, volume 93, pages 293–312, 2005. doi: 10.1109/JPROC.2004.840848. BeBOP/Sparsity project; Section 4.2 describes the register-blocking search.

Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005. doi: 10.1109/JPROC.2004.840301.

Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, 2nd edition, 2008. doi: 10.1137/1.9780898717761.

David A. Harville. Assigning probabilities to the outcomes of multi-entry competitions. *Journal of the American Statistical Association*, 68(342):312–316, 1973. doi: 10.1080/01621459.1973.10482425.

ICMIZER. ICMIZER — tournament poker icm calculator. `https://www.icmizer.com/`, 2024. Commercial Monte Carlo ICM tool; representative of Monte Carlo approaches to ICM equity estimation.

Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, 2nd edition, 1999.

Mason Malmuth. *Gambling Theory and Other Topics*. Two Plus Two Publishing, 1987. Section "Settling Up in Tournaments: Part III" (p. 233) introduces the ICM framework for multi-table tournament equity.

Masatake Mori and Masaaki Sugihara. The double-exponential transformation in numerical analysis. *Journal of Computational and Applied Mathematics*, 127(1–2):287–296, 2001. doi: 10.1016/S0377-0427(00)00501-X.

NVIDIA Corporation. CUDA graphs. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#cuda-graphs`, 2024a. CUDA C++ Programming Guide, Chapter 3.2.8.

NVIDIA Corporation. cuFFTDx: Device-side FFT extensions library. `https://docs.nvidia.com/cuda/cufftdx/`, 2024b. Part of the NVIDIA Math Libraries (MathDx).

Markus Püschel, José M. F. Moura, Jeremy R. Johnson, David Padua, Manuela M. Veloso, Bryan W. Singer, Jianxin Xiong, Franz Franchetti, Aca Gačić, Yevgen Voronenko, Kang Chen, Robert W. Johnson, and Nicholas Rizzolo. SPIRAL: Code generation for DSP transforms. *Proceedings of the IEEE*, 93(2):232–273, 2005. doi: 10.1109/JPROC.2004.840306.

Tysen Streib. New algorithm to calculate ICM for large tournaments. `https://forumserver.twoplustwo.com/15/poker-theory-amp-gto/new-algorithm-calculate-icm-large-tournaments-1098489/`, 2011. Two Plus Two Poker Theory forum.

Lloyd N. Trefethen and J. A. C. Weideman. The exponentially convergent trapezoidal rule. *SIAM Review*, 56(3):385–458, 2014. doi: 10.1137/130932132.

Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. Cambridge University Press, 3rd edition, 2013. doi: 10.1017/CBO9781139856065.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. In *Communications of the ACM*, volume 52, pages 65–76, 2009. doi: 10.1145/1498765.1498785.